Part 2 Brief · Core server

mcp-ops-server · 18 Oct 2026 · PR #3 · Status: Codex review fixed (2x P1, 3x P2), 66 tests, ready to merge

WHAT WAS BUILT

- **Synthetic clinic data** (`seed.py`, fixed seed): 12 providers across 6 specialties (4 need a referral), 40 patients, 2,688 thirty-minute slots over 4 weeks, 42 referrals, 128 pre-booked appointments. The same seed always gives the same IDs.
- **Five tools** over a SQLite store: `search_providers`, `list_open_slots`, `book_appointment`, `cancel_appointment`, `get_referral_status`.
- **66 tests** (up from 3), including races between separate database connections and a real HTTP client running the full find, check, book, cancel flow.

HOW IT WORKS

Layer	Role
tools/	Thin wrappers. Typed inputs (enums, ID patterns, bounded limits) and a TypedDict output for each tool. Store errors become ToolErrors whose message names the next tool to call.
store.py	All business rules. Booking checks: slot open, patient free at that time, approved referral owned by this patient and matching the specialty. One check-then-insert transaction.
SQLite schema	Partial unique index `(slot_id) WHERE status='booked'`: the database itself refuses a second booking, so a cancelled slot can be rebooked but an open one is never booked twice.

WHY IT WAS DONE THIS WAY

- **Rules in the store, constraint in the schema.** The app check gives agents a clear error; the index makes double booking impossible even if a future code path forgets the check.
- **Cancelling needs the owner's patient ID**, and someone else's appointment gives the same "not found" as a fake ID, so the tool can't be used to probe which appointments exist.
- **Deterministic, clock-free data**, so Part 4 eval tasks can name exact slots and referrals. Trade-off: booking past slots is not blocked yet.

WHAT WENT WRONG (LOGGED IN FAILURES.MD)

- **Threads.** The SDK runs tools in worker threads. A mutation test with the store lock removed crashed with `sqlite3.InterfaceError`. Kept the lock and added BEGIN IMMEDIATE.
- **Regex.** pydantic's `\d` matches any Unicode digit, so a patient ID written in Arabic-Indic digits passed validation. Switched to `[0-9]` and added a test.
- **Codex review: 2x P1, 3x P2, all reproduced, then fixed.** A failed COMMIT left the store unable to write; my race test shared one connection, so it never exercised BEGIN IMMEDIATE; two processes starting together both tried to create the schema; APT-100000 could be booked but not cancelled; year-9999 dates crashed. Each fix has a test that fails with the bug put back.

VERIFY IT YOURSELF

```
pip install -e ".[dev]"
ruff check . && python -m pytest -q   # 66 passed
python -m mcp_ops_server              # then call tools at http://127.0.0.1:8000/mcp
```

INTERVIEW TALKING POINT

"Double booking is prevented twice: in application code so the agent gets an actionable error, and as a partial unique index so the database refuses it. The review caught that my race test only proved the Python lock, so I rewrote it across separate connections and showed each test fails when its protection is removed."

NEXT: PART 3, HARDENING

Bearer auth and scopes enforced in the server, rate limits, payload size and timeouts, and a "schema drift" snapshot test that flags any change to a tool schema. **Your action:** merge PR #3, then reply "go".